

Engineering and Architecture Handbook

A controlled snapshot generated from the governed Markdown source. The live knowledge base remains authoritative for current guidance.

Klar release	2.6.0.1
Classification	PUBLIC
Generated	2026-08-10
Implementation baseline	ccc20090c3ed
KB content hash	2a9943544a7daa14
Controlled-copy notice	Verify against the live KB before operational use.

Table of contents

Architecture	3
System Context and Runtime Topology	4
Codebase and Domain Catalog	9
Data	15
Data Architecture, Vault, and Backup	16
Discovery	23
Career Discovery and Ranking	24
Flexible Work and Source Fabric	29
Application Preparation	35
Resume and Application Generation	36
AI and Models	42
AI, Provider, Prompt, and Evidence Architecture	43
Platform	49
Worker API and Network Security	50
Electron Desktop and Local Runtime	54
Engineering	59
Engineering Setup and Change Guide	60

Architecture

2 controlled documents in this section.

System Context and Runtime Topology

Verified architecture boundaries, runtime components, trust zones, and end-to-end data flows for Klar 2.6.0.1.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

This page defines where Klar executes, which component owns each responsibility, and where personal or third-party data can cross a boundary. It is the starting point for design review, incident analysis, and any change that introduces storage, networking, or code execution.

Klar 2.6.0.1 is a local-first React application with two delivery forms:

- a static web application hosted on GitHub Pages; and
- an Electron developer preview that embeds the same renderer and adds a constrained local-model runtime.

The application has no Klar-operated account database and no server-side career-data store. IndexedDB in the active browser or Electron profile is the system of record. A Cloudflare Worker is a bounded network relay for selected job sources and Groq; it is not an application backend.

Context map

Actor or system	Role	Data exchanged	Trust treatment
Person using Klar	Supplies Resume, preferences, decisions, credentials, and export requests	Personal data and career facts	Primary data owner; explicit action is required for external AI and exports
Klar renderer	Runs workflows, validation, ranking, storage, and document generation	Local application state	Trusted application code, but still inside the browser threat boundary
IndexedDB and session storage	Persist workspace data and selected settings or temporary credentials	Structured rows, ciphertext, caches	Device-local; optional vault protects selected content at rest
Cloudflare Worker (k _{lar} -proxy)	Relays allowlisted source retrieval and selected Groq routes	Search parameters, source responses, AI request/response	Separate network trust zone; must not persist user content by design
Public job sources	Return vacancy or route information	Search queries and public listing data	Untrusted external input; normalize and validate before use
Configured AI provider	Performs explicitly invoked extraction, analysis, or writing	Bounded prompt projection and generated response	External processor selected by the user; output is untrusted until validated and reviewed
Electron main process	Enforces desktop navigation, IPC, package trust, and runtime lifecycle	Validated IPC payloads and local inference requests	Privileged boundary; renderer receives no general Node or file access
Local llama-server	Runs a verified model package on loopback	Prompt and generated JSON	Desktop-only, process-isolated, authenticated over ephemeral loopback
Export destination	Receives user-created backup, DOCX, PDF, or ZIP	Selected workspace or application artifact	Outside Klar after download; user controls subsequent retention and sharing

Runtime components

Shared renderer

`src/main.tsx` mounts the React application. `src/App.tsx` owns top-level onboarding, vault gating, mode selection, and workspace routing. The renderer contains the product logic under `src/`; it is not a thin client over a remote API.

The production web build uses the `/klar/` base path. The desktop build uses a relative `./` base so packaged `file://` loading works. The service worker is registered only for production web builds and is not registered in Electron.

Local persistence

Dexie wraps a database named `klar`. Schema version 7 contains 17 stores. The canonical Resume, application packets, searches, results, and tracking state are read and written locally. Optional vault encryption changes the at-rest representation of sensitive stores but does not create a remote sync service. See [Data Architecture, Vault, and Backup](#).

Cloudflare Worker

The Worker has four capabilities: health, Federal Employment Agency (BA) relay, Adzuna relay, Source Fabric retrieval, and the two supported Groq paths. It rebuilds upstream requests from fixed routes and allowlists rather than accepting arbitrary upstream URLs. See [Worker API and Network Security](#).

Electron boundary

The renderer is sandboxed with Node integration disabled. A frozen preload bridge exposes only system information, local runtime lifecycle, generation/cancellation, and redacted diagnostics. The Electron main process validates the sender and every payload before controlling the local model process. See [Electron Desktop and Local Runtime](#).

Principal data flows

1. Resume onboarding and storage

- 1 The person selects a PDF, DOCX, or text file.
- 2 Klar extracts text in the renderer; the original file is not uploaded or retained as a normal workflow artifact.
- 3 If AI extraction is chosen, Klar sends a bounded text projection to the configured provider through the direct or Worker path.
- 4 The response is schema-validated and normalized into Resume schema version 2.
- 5 The person reviews the result before it becomes the canonical Resume.
- 6 Klar writes the canonical row and bounded history locally, in vault ciphertext when the vault is enabled.

2. Career discovery and ranking

- 1 Klar derives a thin matching profile from the canonical Resume at runtime.
- 2 Source adapters retrieve jobs directly or through the Worker.
- 3 Klar validates, normalizes, deduplicates, filters, and ranks results locally.
- 4 Deterministic results are publishable without AI. Provider analysis is optional attention or explanation work and cannot replace deterministic eligibility, confidence, or rank.
- 5 Results and diagnostics are cached locally. See [Career Discovery and Ranking](#).

3. Flexible Work discovery

- 1 Flexible Work preferences supply their own cities, radius, arrangements, roles, and workplaces; no Resume is required.
- 2 A bounded session reads valid cache, runs enabled connectors, accepts independent batches, and terminates at its hard deadline.
- 3 Every accepted opportunity carries source and field-level provenance.
- 4 Duplicates merge in place and official direct application destinations take precedence.
- 5 Search, cache, and connector-health state remain local. See [Flexible Work and Source Fabric](#).

4. Application preparation

- 1 The selected job is copied into a local application packet snapshot.

- 2 Deterministic tailoring proposes a conservative baseline from existing Resume facts.
- 3 Optional AI creates structured proposals constrained by Resume evidence and the job description.
- 4 The person accepts, rejects, or edits each proposed change; unsupported changes are blocked at decision and export boundaries.
- 5 Klar generates DOCX, browser-print PDF, or a ZIP locally. See [Resume and Application Generation](#).

5. Backup and recovery

- 1 Klar assembles a local workspace snapshot.
- 2 The selected mode either excludes credentials, preserves existing vault ciphertext, encrypts sensitive data with a backup password, or-after an exact warning confirmation-exports readable sensitive data.
- 3 Import validates format and digest, authenticates encrypted material, migrates supported historical schemas, and atomically replaces the database.

Trust boundaries and invariants

Boundary	Invariant
Renderer to IndexedDB	All persisted product state has an explicit store and migration path; sensitive plaintext stores are empty while the vault is enabled
Renderer to Worker	Only configured Worker routes are used; source data and AI responses remain untrusted input
Worker to source	The upstream host/path is fixed or allowlisted and redirects are revalidated
Renderer to AI	Invocation is explicit, prompt input is minimized, response schemas are validated, and generated claims require evidence and human review
Renderer to Electron main	Only the narrow preload contract is available; sender, frame, origin, types, sizes, and lifecycle are checked again in main
Electron main to runtime	Only verified compatible packages execute; loopback is ephemeral and authenticated; the renderer never receives the bearer credential
Klar to downloaded file	Export occurs only on user action; after download, protection and retention are the recipient's responsibility

Availability and deployment implications

- Web application availability depends on the static host, cached application assets, configured Worker, and third-party sources. An installed service worker can preserve parts of the shell, but Klar does not claim complete offline discovery.
- The static app does not continue searches while closed. A search exists only while its active browser or desktop session runs.
- A Worker failure reduces external source and Groq connectivity; it does not delete local state or prevent deterministic local workspace use.
- A third-party connector failure must be isolated from other connectors and produce an honest partial or limited state.

- Desktop local inference can remain usable without an internet connection only after a trusted runtime and compatible model package are installed; normal product generation is not yet routed to that provider in 2.6.0.1.

Review triggers

Review this page when any of the following changes: a new remote service, storage location, IPC method, Worker route, model runtime, service-worker behavior, background task, account/sync feature, or export destination.

Related controls are documented in [Security, Privacy, and Threat Model](#) and operational responses in [Operations Runbooks](#).

Codebase and Domain Catalog

Ownership map for Klar source directories, runtime entry points, domain responsibilities, and governing invariants.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

This catalog explains why each major code area exists and which invariant it protects. It is an orientation and change-impact document, not a substitute for reading the implementation and tests. The repository, configuration, and current verification evidence remain authoritative under [Source of Truth](#).

Repository map

Path	Responsibility	Governing invariant
src/main.tsx	Renderer bootstrap, locale provider, service-worker registration	Service worker runs only in production web, never desktop
src/App.tsx	Top-level vault gate, onboarding, Career/Flexible mode routing, persistent mounted searches	Flexible Work never requires a Resume; Career discovery does
src/types.ts	Shared product contracts for jobs, preferences, matches, tracking, search, and Flexible Work	Shared shapes must remain compatible across storage, UI, source, and test boundaries
src/db/	Dexie schema, migrations, table contracts, database wipe	Every persisted shape has an explicit store and migration strategy
src/storage/	Vault-aware accessors for career data	Callers should not bypass the encrypted/plaintext routing decision
src/crypto/	AES-GCM envelopes, passphrase derivation, vault state transitions	Authentication precedes use; sensitive plaintext and active vault ciphertext must not coexist after a completed transition
src/backup/	Backup formats, integrity, migration, import/export modes	Credential inclusion is explicit and atomic import validates before replacement
src/resume/	Canonical Resume schema, normalization, persistence, completeness, extraction, and design evaluation	Resume schema version 2 is the sole career-fact authority
src/parse/	Client-side file text extraction and legacy parsing helpers	Files are parsed locally and untrusted content is bounded before downstream use
src/sources/	Career source adapters, registry, normalization, acquisition, and source health	Individual source failure cannot collapse discovery; all rows normalize before use
src/search/	Career saved-search identity and diagnostics coordination	"New" is based on durable identities; the first run establishes a baseline
src/match/	Local filters, deterministic ranking, requirements, optional AI attention	Eligibility and rank are deterministic and reproducible; AI does not overwrite them
src/regions/	Supported market configuration and source availability	Market behavior is data-driven and explicit rather than inferred from the Resume
src/flexible/	Resume-free session orchestration, taxonomy, relevance, deduplication, cache, resilience, and connector registry	Search terminates, unknown remains unknown, and provenance survives every transformation
src/flexible/connectors/	Source Fabric connector execution and parsing	Retrieval obeys registry capabilities, bounds, validation, and fallback policy
src/application/	Evidence, writing contracts, workspace routing, generation state, and readiness	Generated content is a proposal and unsupported claims cannot pass export readiness
src/packets/	Persistent per-job application workspaces and artifact provenance	Each language and artifact retains its own provenance and history
src/export/	Resume, cover-letter, spreadsheet, PDF/print, and ZIP production	Export uses reviewed current state and stable, versioned format contracts

Path	Responsibility	Governing invariant
src/ai/	Provider-neutral request, capability, adapter, registry, and validation foundation	Providers are explicit; the registry never silently falls back
src/llm/	Current production provider client, prompt construction, extraction, matching, and tailoring	Structured responses are validated at the consumer; budgets and retry counts are bounded
src/settings/	Provider settings and credential access	Credentials use session, local setting, or credential vault according to the selected retention mode
src/observability/	Content-free product events and diagnostics	Operational records contain fixed safe fields, not Resume, job, prompt, or credential content
src/support/	Redacted issue-report preparation	Reports are previewed and user-submitted; Klar does not silently transmit them
src/onboarding/	Local setup-state and work-mode progression	Setup is recoverable and Flexible Work remains available without a Resume
src/dashboard/, src/tracker/	Local activity summaries and application tracking	These views use local authoritative packet/tracker data
src/i18n/	English/German interface strings and locale mechanics	Locale changes presentation, not evidence or stored career facts
src/ui/	Product surfaces and interaction orchestration	UI must preserve domain gates rather than reimplementing or weakening them
worker/src/	Cloudflare relay routes and network enforcement	Worker is a finite allowlisted relay, never a general proxy
desktop/	Electron main/preload, model trust, runtime lifecycle, packaging, and signing configuration	Privileged capabilities remain outside the renderer and packages execute only after trust verification
model/	Model manifests, schemas, experiments, benchmark tools, and evidence	Experimental output is not release evidence; base and adapter claims require the defined gates
scripts/	Release, registry, bundle, ranking, writing, and document QA automation	Automated gates must be deterministic and fail closed on invalid inputs
qa/	Packaged desktop smoke and fuse verification	Packaged artifacts, not just source, must preserve the security posture
test/	66 repository test files executed by the custom sorted runner	New behavior needs a test at the closest contract boundary and a regression test for defects
.github/workflows/	Web deployment and desktop-preview CI	Deployment follows the repository gates; local success alone is not release approval
knowledge-base/	Governed corporate documentation website and content	Documentation distinguishes current, target, and historical statements

Domain dependency direction

The intended dependency direction is:

- 1 contracts and pure domain rules;
- 2 storage, source, provider, and runtime adapters;
- 3 workflow orchestration;
- 4 UI presentation;
- 5 deployment and packaging.

UI components may coordinate domain functions, but they must not become the only location for ranking, evidence, encryption, trust, or network-security rules. Worker and Electron checks must remain authoritative in their privileged processes even if the renderer validates first.

Major business objects

Object	Authority	Purpose and lifecycle
ResumeData	Canonical Resume row current or vault content	Reviewed career facts; normalized at import/edit and versioned through bounded snapshots
Profile	Derived from ResumeData	Thin compatibility/matching projection; never a second career-fact authority
Preferences	Current local row or vault content	Career and Flexible Work constraints, languages, markets, and optional contact data
NormalizedJob	Source normalization output and local caches	Common vacancy/open-entry representation, including identities and provenance
RankingSnapshot	Deterministic ranking pipeline	Reproducible score, features, constraints, explanation, hashes, and version metadata
SearchSession	Flexible Work runtime state	One cancellable, deadline-bound query with independent connector attempt states
PacketRow	Packet store or vault content	Per-job preparation workspace with job snapshot, languages, decisions, generation, and export history
CipherEnvelope	Vault/backup boundary	Versioned AES-GCM authenticated ciphertext with salt and IV
Connector definition	Career/Source Fabric registry	Retrieval capabilities, allowlist, parser version, budgets, validation, fallback, health, and verification state
Model package manifest	Desktop package directory	Signed declaration of model files, compatibility, provenance, runtime, platform, and prompt contract

Entry points and execution ownership

Entry point	Executes in	May do	Must not do
src/main.tsx	Browser/Electron renderer	Mount application and web service worker	Acquire Node or arbitrary device privileges
worker/src/index.ts	Cloudflare Worker	Route, bound, and relay approved requests	Store workspace content or fetch a caller-supplied URL
desktop/main.mjs	Electron main	Create trusted window, validate IPC, manage runtime	Expose generic filesystem, shell, or process access
desktop/preload.mjs	Isolated preload	Publish frozen narrow bridge and prevalidate payloads	Expose Electron IPC primitives or secrets
scripts/run-tests.mjs	Node development/CI	Execute sorted repository tests	Stand in for production build, packaged smoke, human, or signing gates
Vite build	Node development/CI	Type-check and produce static renderer assets	Prove third-party source health or human writing quality

Change-impact rules

- A shared type change requires checking storage, backup, normalization, UI, fixtures, and historical migration behavior.
- A Resume schema change requires its own schema version, normalization, migration snapshot, vault/backup compatibility, exporter review, and evidence tests.
- A connector change requires registry, Worker allowlist, parser contract, fallback, health, cache, provenance, and failure-isolation review.
- A prompt or generated-artifact change requires input projection, schema, evidence, locale, retry, provenance, export, machine checks, and-where claims change-human gate review.
- A desktop IPC change requires preload and main validation, sender checks, CSP/navigation impact, diagnostics redaction, lifecycle tests, and packaged smoke review.
- A Worker route change requires method, CORS, allowlist, redirects, size, content type, timeout, error disclosure, log, and abuse-control review.

The procedural version of these rules is in [Engineering Setup and Change Guide](#). Release evidence is defined in [Test Strategy and Release Gates](#).

Known catalog drift

The header comment in `src/db/db.ts` still says six stores hold all state. That was true for schema version 1; schema version 7 has 17 stores. The executable schema is authoritative. This inconsistency is tracked in [Known Gaps and Risk Register](#).

Data

1 controlled documents in this section.

Data Architecture, Vault, and Backup

Current Klar persistence model, Dexie schema history, encryption boundary, backup formats, recovery controls, and retention rules.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

This page defines the authoritative local data model and the controls that prevent schema changes, encryption transitions, or backup operations from silently losing or exposing user data.

Storage principles

- 1 **Local system of record.** Product state is stored in the browser or Electron profile, not in a Klar account service.
- 2 **One career-fact authority.** ResumeData schema version 2 is canonical. Thin profile data is derived when needed.
- 3 **Optional authenticated encryption.** The vault protects selected content and credentials at rest; it does not claim protection against compromised code or an unlocked device.
- 4 **Explicit portability.** Backup and export are user actions with modes that make credential and readable-data treatment clear.
- 5 **Atomic transitions.** Vault enable/disable and backup import authenticate and validate before replacing or deleting authoritative rows.
- 6 **Unknown is not invented.** Missing fields remain absent through normalization and migration.

Dexie database

The database name is `klar`; the current Dexie schema version is 7.

Store	Primary key	Contents	Sensitive-content treatment
settings	key	Locale, feature/config values, provider configuration, operational events	Remains outside content ciphertext; credential helpers control secret rows separately
profiles	id	Historical thin-profile compatibility store	Cleared by v5 migration; current Profile is derived

Store	Primary key	Contents	Sensitive-content treatment
preferences	id	Career and Flexible Work preferences	Moved into vault content while enabled
jobs	queryKey	Career result cache	Moved into vault content while enabled
matches	cacheKey	Career match results	Moved into vault content while enabled
tracked	jobId	Application tracker rows	Moved into vault content while enabled
dashboard	id	Personal dashboard row	Moved into vault content while enabled
vectors	jobId	Local embedding cache	Moved into vault content while enabled
savedSearches	id	Career searches and seen identities	Moved into vault content while enabled
vault	id	Content and optional credential ciphertext envelopes	Ciphertext authority while enabled
resumes	id	Canonical Resume row	Moved into vault content while enabled
resumeHistory	id	Bounded Resume snapshots	Moved into vault content while enabled
resumeDrafts	id	Recoverable Resume draft	Moved into vault content while enabled
flexibleSearches	id	Saved Resume-free searches	Moved into vault content while enabled
flexibleCache	queryKey	Validated Flexible Work opportunity cache	Remains a plaintext operational/cache store
connectorHealth	connectorId	Content-free failure, latency, breaker, and kill-switch state	Remains plaintext; should contain no personal content
packets	id	Persistent application workspaces and provenance	Moved into vault content while enabled

“Plaintext” means not protected by the optional vault; it is still inside the browser's origin-scoped storage. Operating-system account, disk, browser-profile, and device protections remain relevant.

Schema and migration history

Dexie version	Product era	Change	Migration behavior
1	Original	Six stores: settings, profiles, preferences, jobs, matches, tracked	Initial schema
2	Early dashboard	Adds dashboard and vectors	Additive
3	Saved search	Adds saved searches	Additive
4	2.2 vault	Adds vault; removes legacy <code>rawText</code> from profile rows	Vault remains opt-in; upgrade deletes the debug field
5	2.3 canonical Resume	Adds Resume, history, draft; converts legacy rich Resume or latest Profile; clears profiles, match, and vector caches	One atomic transaction and migration snapshot
6	2.4 Source Fabric	Adds Flexible Work searches, cache, and connector health	Additive
7	2.5 Application Quality	Adds packet store	Deliberately additive; no Resume migration folded into it

Migration code must be idempotent within Dexie's upgrade semantics, preserve unsupported historical input until validation succeeds, and invalidate any derived cache whose inputs or meaning changed.

Canonical Resume retention

The current Resume row uses `ID current`; the onboarding draft uses `onboarding`. Resume history retains at most 10 snapshots and removes snapshots older than 90 days. Snapshot reasons include edit, reupload, migration, manual save, and restart/recovery. This is an edit-recovery mechanism, not indefinite records management.

Saved-search seen identities are capped at 5,000 and expire after 180 days. Packet versions are bounded to five per packet and export history to 20 entries. Operational events are bounded to 100 browser events and desktop diagnostics to 200 in-memory events.

Vault cryptography

Primitive and envelope

`src/crypto/resumeCrypto.ts` uses Web Crypto:

- PBKDF2 with SHA-256;
- 210,000 iterations;
- a random 16-byte salt;
- AES-GCM with a 256-bit key;
- a random 12-byte IV; and
- envelope version 1 containing `v`, `salt`, `iv`, and `ct`.

AES-GCM authentication detects an incorrect passphrase and ciphertext modification before plaintext is accepted. The salt and IV are non-secret parameters and travel with the ciphertext.

Logical separation

The vault row has separate authenticated envelopes:

- **content envelope**, logical schema version 3: canonical Resume/history/draft, preferences, jobs, matches, tracker, dashboard, vectors, saved searches, Flexible Work searches, and packets, with reserved collections for original files and knowledge-base content;
- **credential envelope**, logical schema version 1: remembered Groq credential and the atomic Adzuna credential pair.

This separation allows a standard backup to retain protected content while excluding credential ciphertext byte-for-byte.

State transitions

Enabling encryption requires a passphrase of at least 12 characters and the required acknowledgements. Klar assembles the sensitive snapshot, encrypts and authenticates it, commits the vault row, and deletes the corresponding plaintext rows in one database transaction. Unlock keeps plaintext in memory and migrates any supported legacy Flexible Work data into the content envelope. Vault mutations are serialized; an epoch guard prevents a queued write from committing after lock.

Disabling encryption decrypts and authenticates first, writes plaintext stores, and removes the vault row atomically. Lock clears the in-memory unlocked state. Klar cannot recover a forgotten passphrase.

Security boundary

The vault is designed to reduce exposure from casual IndexedDB inspection, copied browser profiles, and device-at-rest access without the passphrase. It does not protect data from malicious code running in the same unlocked origin, a compromised operating system, screen capture, keylogging, or a person using an already unlocked Klar session. See [Security, Privacy, and Threat Model](#).

Credentials

Groq credential storage depends on the “remember” choice and vault state:

- remembered with vault enabled: credential envelope;
- remembered without vault: local setting controlled by the credential helper;
- not remembered: session storage or in-memory state.

Adzuna credentials are a pair and are accepted only when both application ID and key are present. Worker-provided credentials and user-provided credentials are not partially combined.

Provider endpoint/model configuration is not itself treated as a secret and stays in settings. Custom endpoint configuration may reveal organizational infrastructure and should still be treated as non-public configuration in support reports.

Backup contract

Current backups use format identifier `k1ar-backup`, schema version 6, and include Klar release metadata.

Mode	Sensitive data	Credentials	Required action
Standard	Plain workspace when vault is off; existing content ciphertext when vault is on	Secret setting rows and credential ciphertext removed	Normal export confirmation
Complete encrypted	If vault is off, selected sensitive workspace and credentials are encrypted with a backup password of at least 12 characters; if vault is on, existing encrypted envelopes are copied	Included only inside authenticated ciphertext	Password/explicit complete-backup choice
Readable sensitive export	Decrypted workspace content	Depends on the explicitly selected readable payload; should be treated as highly sensitive	Exact confirmation text EXPORT DECRYPTED DATA and unlocked vault when enabled

Every backup carries a SHA-256 digest calculated over the JSON representation without the integrity field. This detects accidental modification; it is not a digital signature and does not establish who created the file.

Import and recovery

Import follows a validate-before-replace sequence:

- 1 parse JSON and validate the format, version, mode, and required top-level structure;
- 2 verify the SHA-256 digest when present;
- 3 decrypt and authenticate encrypted material before database mutation;
- 4 migrate supported historical formats, including raw legacy v1/v2/v2.1, schema v4 from 2.2, and schema v5 from 2.3-2.5;
- 5 validate primary structure and key expectations;
- 6 replace the workspace in one Dexie transaction; and
- 7 reload the application so all readers observe the imported authority.

Structural validation is intentionally bounded and does not deeply validate every nested property of every row. Imports from unknown tools or edited JSON therefore remain untrusted. Use the controlled procedure in [Operations Runbooks](#).

Data deletion

The application-level wipe clears all 17 stores. Browser or Electron profile deletion is an additional platform-level removal option. Downloaded exports, backups, provider-side request retention, and copies shared with employers are separate systems and are not deleted by clearing Klar.

Change control checklist

Any persistent-data change must answer:

- What is the single authority after the change?
- Is the shape sensitive, operational, cache-only, or a credential?

- How does it behave with the vault locked, unlocked, enabled, and disabled?
- Is it included in each backup mode and historical import path?
- What is the migration, rollback, cache invalidation, and failure atomicity story?
- What retention bound and deletion path apply?
- Which tests prove new, upgrade, backup, wrong-password, corrupt-input, and interrupted-operation behavior?

Implementation procedures are in [Engineering Setup and Change Guide](#); open limitations are tracked in [Known Gaps and Risk Register](#).

Discovery

2 controlled documents in this section.

Career Discovery and Ranking

Career-source acquisition, normalization, filtering, deterministic ranking, optional AI attention, and diagnostic contracts.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

Career discovery turns public job-source responses and reviewed Resume facts into a deterministic, explainable candidate list. Its purpose is to focus attention without claiming to decide employability, suitability, or hiring outcome.

The ranking pipeline is local-first. AI is optional, bounded, and subordinate to deterministic eligibility, rank, and posting-confidence results.

Preconditions and inputs

Career mode requires:

- a canonical Resume from which Klar derives a matching Profile;
- current career preferences and a search query;
- a selected supported region; and
- at least one enabled source path.

The Resume location informs Career behavior only. It is not imposed on Flexible Work. Completeness checks are structural; they do not certify career quality or truth.

Acquisition pipeline

The current sequence is:

- 1 build the regional source plan;
- 2 run active source adapters in parallel with failure isolation;
- 3 normalize every source row to NormalizedJob;
- 4 deduplicate exact, identity, content, and fuzzy matches while retaining a lso_on provenance;
- 5 apply local employment, hidden-employer, recency, and distance filters;
- 6 apply Career role, market, and seniority relevance gates;

- 7 lazily enrich BA rows whose description is empty;
- 8 repeat relevance evaluation after enrichment;
- 9 compute deterministic requirements, ranking, explanations, and posting confidence;
- 10 publish all deterministic candidates; and
- 11 optionally allocate AI attention without changing the deterministic order.

Explicit mismatches are separated with reasons. Unknown data is generally retained rather than converted into a negative fact.

Career source catalog

Source	Identifier	Transport	Search behavior	Important constraints
Bundesagentur für Arbeit (BA)	ba	Cloudflare Worker	Server-side search, 50 rows per page	Search and detail enrichment use separate upstream contracts without a public compatibility guarantee
Arbeitsnow	arbeitsnow	Direct browser CORS	Fetches first two public pages and narrows locally	Not a true query API; practical acquisition is roughly bounded by those pages
Adzuna	adzuna	Cloudflare Worker	Region-specific API, 50 rows per page	Needs a complete user credential pair or complete Worker credentials
Greenhouse	greenhouse	Direct public ATS	Per-company board retrieval	Company registry controls fan-out; failure isolated per company
Lever	lever	Direct public ATS	Per-company board retrieval	Same registry and isolation contract
Ashby	ashby	Direct public ATS	Per-company board retrieval	Same registry and isolation contract

ATS retrieval uses bounded concurrency of six companies. The current registry exports 47 verified German entries and 162 DACH candidate entries, combined as 209 entries. The current fetch path includes both verified and candidate entries. This is implementation truth, but it is also a data-quality and load-control gap because candidate status does not currently prevent production retrieval.

Region and source planning

Supported regions are Germany, Austria, Switzerland, the Netherlands, Luxembourg, and Liechtenstein. Region configuration determines currency and source availability:

- Germany: BA, Adzuna, and direct sources;
- Austria, Switzerland, and the Netherlands: Adzuna and direct sources;
- Luxembourg and Liechtenstein: direct sources only.

The normal plan starts with BA, Arbeitnow, and ATS sources. Adzuna becomes a default source only when a Worker URL is configured. Users can still control supported source choices through product settings. A source being configured does not prove it is healthy or complete.

Normalization and identity

Every adapter maps source-specific data into the same job contract before filtering. At minimum, downstream behavior depends on stable source identity, title, company, location, dates, description, application URL, employment information, and source provenance where published.

Deduplication combines:

- native stable IDs;
- canonical application/source identities;
- normalized content fingerprints; and
- fuzzy job identity when exact identifiers differ.

When rows merge, provenance is preserved and all known sources appear in a `also_on`. A direct or higher-quality representation may improve the canonical row, but the merged row must not pretend the other observation never existed.

Saved-search “new since last check” uses durable identities, not the transient display rank. The first run creates the baseline and does not label every existing job as new. Seen identity history is bounded to 5,000 values with a 180-day retention horizon.

Deterministic eligibility

The requirements evaluator records facts for:

- work authorization;
- location;
- language;
- employment arrangement;
- working hours;
- start date;
- certification; and
- user-defined dealbreakers.

Facts carry known, unknown, satisfied, or mismatch meaning rather than forcing missing listing data into a negative conclusion. A known hard mismatch can exclude a job. An unknown requirement remains visible for review.

Local relevance gates protect the candidate set from unrelated markets, roles, and seniority levels. A hard-filtered row cannot re-enter through AI enrichment or reranking.

Ranking version 2

Current contracts are:

- ranking: ranking-v2.6.0;
- requirements: requirements-v2.6.0; and
- explanation: ranking-explanation-v2.6.0.

The core-fit weighted dimensions are:

Dimension	Weight	Meaning
Role fit	30%	Alignment between requested/Resume role and the vacancy
Seniority fit	15%	Compatibility of demonstrated and requested level
Required-skill coverage	25%	Evidence-backed coverage of explicit requirements
Preferred-skill coverage	5%	Evidence-backed coverage of non-mandatory preferences
Domain transfer	15%	Transferability of demonstrated experience into the vacancy context
Evidence depth	10%	Strength and specificity of supporting Resume evidence

Caps constrain scores when there is inadequate relevance, domain transfer, seniority alignment, missing requirements, or a known mismatch. This prevents a strong soft signal from compensating for a material constraint.

Preference fit is separate from core fit. Salary, location, work mode, and contract preferences use relative weights of 20, 35, 25, and 20 and can adjust the score by no more than eight points. Posting confidence is also separate: source quality 30%, completeness 30%, freshness 30%, duplicate corroboration 10%. The final ranking applies a penalty equal to 20% of the posting-confidence deficit.

The outer legacy modelVersion remains local-v2.3 for cache/backward compatibility, while the embedded snapshot names ranking-v2.6.0. Consumers must use the ranking snapshot version when interpreting current features and explanations.

Reproducibility contract

Each deterministic snapshot uses schema version 1 and records:

- an input hash;
- a candidate-set hash;
- evaluation time;
- deterministic rank and score;
- features and requirement facts;
- human-readable explanations;

- algorithm contract versions; and
- historical/unversioned markers when old data lacks current provenance.

For identical normalized inputs and configuration, feature evaluation and rank must be deterministic.

Time-dependent source freshness is part of the input context and can legitimately change results between runs.

Optional AI attention

The full deterministic result set is not capped. The `candidateLimit` value of 40 limits automatic AI priority, not visible results. AI batching is five jobs per request; the legacy shortlist size is eight; default LLM reranking is off.

There are two supported attention patterns:

- enrich one job the person opens, then cache that explanation; or
- in compatibility mode, automatically enrich a bounded top set.

If AI is unavailable, malformed, over budget, or missing a row, the deterministic local result remains. Only valid provider results are cached. Provider scores and prose cannot overwrite deterministic eligibility, posting confidence, or order.

Diagnostics and honest empty states

Search diagnostics reconcile:

- sources requested, succeeded, failed, or skipped;
- raw rows;
- duplicate merges;
- local filter and relevance reasons;
- candidate count;
- local and AI analysis counts;
- provider failures; and
- final visible count.

The zero-result reason must distinguish “healthy sources returned no matching jobs” from “coverage was insufficient to conclude that.” Errors and reports use content-free safe fields. Source health is local operational metadata, not proof of complete source inventory.

Quality gates

Ranking release claims require both automated invariants and the independent human evaluation defined in [Test Strategy and Release Gates](#). The current repository records the human ranking gate as **HOLD** until real, independent, professionally bilingual review evidence is complete. Synthetic and machine-generated fixtures cannot satisfy that gate.

Known source-verification, version-label, and gate gaps are tracked in [Known Gaps and Risk Register](#).

Flexible Work and Source Fabric

Resume-free discovery architecture, connector registry, bounded search sessions, taxonomy, provenance, fallback, cache, and health behavior.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

Flexible Work gives students and other users a first-class discovery path for schedule-compatible work without requiring a Resume. Source Fabric exists because no single durable API covers the required employer and job landscape. It combines bounded, source-specific retrieval contracts behind one normalized opportunity model.

This page describes the executable 2.6.0.1 baseline. The July 2026 Source Fabric roadmap supplied the design intent, but code, configuration, and tests determine current behavior.

Product separation

Flexible Work owns its own:

- cities and radius;
- employment arrangements;
- role families;
- workplace preferences;
- optional keywords;
- saved searches and seen identities; and
- lightweight preparation path.

The canonical Resume and its location are not required inputs. A person with a Resume can still switch modes; one mode does not hide or replace the other.

Opportunity contract

Source Fabric extends the shared normalized job shape with:

- kind: vacancy or open-entry route;
- source identities and canonical ID;
- employer family, brand, and connector identity;

- locations and optional coordinates;
- employment arrangements, weekly hours, and schedule signals;
- role families and workplaces;
- description, salary, posted/expiry/first-seen/last-seen/last-verified dates;
- application URL;
- source and extraction confidence; and
- per-field provenance.

An open-entry program is not represented as a fabricated vacancy. It must remain labelled as an open application or candidate-pool route. Unknown published facts remain unknown; inferred classifications are distinguishable from published facts.

Connector types

Type	Intended retrieval	Example role
API	Supported public JSON API or ATS response	BA, Adzuna, Arbeitnow, supported ATS
Feed	Employer-provided RSS or Atom	Durable employer publication feed
Sitemap	Canonical URL discovery plus structured job detail	Sitemap and JobPosting JSON-LD
Portal	Bounded public listing/search manifest	Structured employer career portal
Open entry	Official program/city verification and route	Courier, restaurant, or general application path
Federated	Parent, regional, franchise, and fallback composition	Employer groups with distributed hiring

The registry contains allowlisted hosts and paths, parser version, query capability, pagination, attempt timeout, retry/cache/response bounds, content types, field coverage, workplaces, fallback, health, and verification metadata. HTML layout parsing is not a general-purpose connector contract.

Current registry baseline

The runtime registry contains 36 enabled connector definitions: 24 top-level connectors and 12 member connectors. It represents 21 employer families, including major grocery, retail, drugstore, logistics, delivery, restaurant, café, and hotel groups.

Connector-type distribution is currently 11 API, 2 feed, 2 sitemap, 12 portal, 4 federated, and 5 open-entry definitions. Three baseline connectors are marked verified; 33 definitions are marked candidate.

The code currently enables and runs candidate connectors; `buildFabric` does not exclude a connector because its verification field is candidate. Registry comments and the original roadmap require verification before production support claims, so current execution is broader than verified-support evidence. Do not describe the registry count as 36 verified direct integrations.

Search-session contract

Each search creates one local session with a stable ID, normalized input snapshot, root cancellation signal, start/deadline, source-attempt states, canonical opportunity map, ordered results, pages, and diagnostics.

Current timing policy:

Control	Value
Connector attempt timeout	Clamped to 10-15 seconds
Maximum retry count	2 after the initial attempt
Hard logical deadline	60 seconds
Page size	20 unique opportunities
First publish threshold	10 unique opportunities
Low-supply publish deadline	8 seconds
Default connector concurrency	6

Retries are for idempotent transient failures only. Invalid request, authentication, schema, content-type, blocked/CAPTCHA, and removed-job responses are terminal for that attempt. A 429 is retryable only when its delay fits the remaining search budget.

Connectors produce independent batches. Klar does not wait for every source before publishing. The session ends in `complete`, `partial`, or `limited`; cancellation and the deadline must stop loading indicators and unfinished work.

Ordering and pagination

The session preserves stable append order after first publication. Later evidence may enrich a duplicate card in place, and a direct employer application URL can replace an aggregator link. Vacancies are placed before route cards during initial ordering.

The roadmap calls visible page assignments “frozen.” The current implementation chunks the evolving ordered array and permits in-place duplicate enrichment. Membership is intended to remain stable after append, but the code does not persist a separate immutable page-assignment ledger. Treat strict frozen-pagination behavior as partially implemented and verify it whenever session ordering changes.

Taxonomy

Classification is deterministic and multilingual across independent dimensions:

- employment: minijob, part-time, working student, temporary, seasonal, weekend, evening, night;
- role: shelf stocking, cashier, sales, picking/packing, warehouse, parcel sorting, delivery, kitchen, counter service, service, cleaning, housekeeping, reception, event work, customer service; and
- workplace: supermarket, retail store, drugstore, warehouse, parcel hub, restaurant, café, hotel, delivery, event venue.

Evidence weights are title 3, employer 2, and description 1; the normal acceptance threshold is 2. Career-like role and employment signals are damped to 0.2 so an isolated broad keyword does not dominate. Workplace evidence is not damped. Explicit published employment terms win over inference.

Each inferred value records evidence, method, confidence, and provenance. The optional local neural suggestion is off by default and may only add a low-confidence suggestion below 0.6; it cannot override explicit published terms.

Relevance gate

The flexible gate rejects a row when positive evidence shows:

- the wrong location;
- a clearly Career-oriented title outside the flexible intent;
- salary above the flexible thresholds (more than 45,000 yearly, 3,750 monthly, or 45 hourly in the configured interpretation);
- no flexible-work signal; or
- explicitly incompatible employment.

Unknown values remain eligible for review. Open-entry routes face the location gate but are not judged as if they were fully described vacancies.

Deduplication and provenance

Every batch is validated and merged into a canonical map. Duplicate resolution must preserve all source observations, field-level provenance, first-seen time, and last-verification time. A later source can improve fields without changing the historical fact that earlier sources observed the opportunity.

Application destination priority favors a direct employer route over an aggregator when both describe the same opportunity. A route card must never silently replace a concrete vacancy.

Cache and freshness

Flexible cache has a normal 30-minute lifetime. Only normalized valid records are cached. Cache reads validate minimum required fields and respect `validThrough` for vacancies. Open-entry routes use separate freshness semantics. Revalidation preserves the original `firstSeenAt` and updates verification metadata; cached content must not be relabelled as newly posted.

If no Worker URL is configured, the current hook can supply deterministic fixtures as the live UI input and sets `usingFixtures=true`. This supports offline development and QA, but a production deployment without the Worker could display synthetic opportunities unless the deployment and UI make that state unmistakable. This is a high-priority honesty/configuration gap.

Resilience, health, and fallback

The fallback ladder is:

- 1 valid local/session cache;
- 2 baseline source inventory;

- 3 direct employer connector;
- 4 employer-filtered BA results plus the official route;
- 5 an official open-entry route; or
- 6 an official employer search destination.

Connector health stores content-free counts, latency, timestamps, schema failures, breaker state, and a manual kill switch. The breaker opens after four consecutive failures and uses a five-minute cooldown. Search skips killed connectors and open circuits. The design calls for a low-frequency canary; current persistence does not model a distinct canary state, and a connector is simply eligible again after cooldown.

Fallback success contributes to the connector observation. Operations must distinguish “direct integration healthy” from “a usable fallback was returned.”

Source security

Fabric retrieval is delegated to the Worker for allowlisted remote hosts. It rebuilds HTTPS URLs, validates paths and redirects, limits body size and accepted content types, rejects listed private IPv4 ranges, and does not execute source JavaScript. Parsing and sanitization remain untrusted-input boundaries. See [Worker API and Network Security](#).

Operational interpretation

- A registry entry is a configured capability, not proof of current inventory or contractual support.
- A successful fallback is usable coverage, not proof that the direct parser works.
- A cached opportunity is a last-verified observation, not a guarantee that an employer still accepts applications.
- A search with required source failures must say partial or limited, not “no jobs found.”
- Source health and operational events must remain content-free.

Runbooks are in [Operations Runbooks](#). Verification and deployment gates are in [Test Strategy and Release Gates](#). Open gaps are consolidated in [Known Gaps and Risk Register](#).

Application Preparation

1 controlled documents in this section.

Resume and Application Generation

Canonical Resume model, import and review, evidence-bound tailoring, packet state, provenance, document contracts, and export gates.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

Klar prepares application material from user-reviewed career facts while preventing generated prose from silently becoming asserted truth. The canonical Resume is the source of facts; AI and deterministic transforms may propose presentation changes, but the person remains the approver.

Canonical Resume contract

ResumeData schema version 2 is the sole authoritative career-fact model. It includes:

- contact details and links;
- summary;
- experience entries with stable IDs, bullets, and evidence references;
- education;
- grouped skills;
- languages;
- projects;
- certifications;
- evidence records; and
- optional review time.

`normalizeResume` trims and regularizes input, supplies stable internal identifiers where needed, and links evidence without creating unsupported career facts. A thin `Profile` is derived at runtime for matching and compatibility. It must not be independently edited or persisted as a competing source of truth.

Completeness is structural: it can identify missing sections or fields, but it cannot judge whether experience is impressive, accurate, or sufficient for a role.

Import and extraction

Supported input paths are PDF, DOCX, and plain text. PDF text is extracted in the browser through a dynamically loaded PDF.js worker; DOCX text is extracted with Mammoth; text files are read directly. The normal workflow does not upload or retain the original document.

AI-assisted Resume extraction:

- sends at most 14,000 characters of extracted text;
- requests a strict structured schema where the provider supports it;
- instructs the model to copy facts and not invent them;
- sanitizes and normalizes partial nested rows; and
- presents the result for review before it becomes canonical.

The legacy thin-profile extraction path is separately bounded at 12,000 characters. New work should target the canonical Resume path.

Persistence and history

The current Resume ID is `current`; the onboarding draft ID is `onboarding`. An accepted edit or replacement creates bounded history. Klar keeps at most 10 snapshots and removes snapshots older than 90 days. Snapshot reasons distinguish edit, reupload, migration, manual, and restart recovery.

While the vault is enabled, current Resume, draft, and history live inside authenticated content ciphertext. See [Data Architecture](#), [Vault](#), and [Backup](#).

Application packet

Each selected job has a packet ID of the form `{career | flexible} : {jobId}`. A packet contains:

- an immutable-enough job snapshot for application context;
- independent English and German workspaces;
- Resume tailoring decisions and evidence findings;
- cover-letter and recruiter-message content;
- notes, readiness, and generation-stage recovery state;
- per-artifact format and generator provenance;
- bounded packet versions; and
- bounded export history.

The job snapshot protects the preparation workspace from later source-cache changes. Language states are independent: generating German content must not silently relabel or overwrite English content.

Packet versions are capped at five and export history at 20. On startup, any generation stage left in flight is marked interrupted so the user sees recoverable state rather than an endless spinner.

Tailoring model

Deterministic baseline

The baseline preserves original Resume facts and reorders or selects existing content to emphasize job-relevant evidence. It does not manufacture achievements, employers, dates, qualifications, or metrics.

AI proposal

AI tailoring returns a structured edit proposal, not a replacement Resume. Each proposed change is associated with original content and evidence. The consumer validates the schema, indices, evidence references, and audit record. A malformed result receives at most one targeted regeneration attempt; `maxTailoringAttempts` is 2 total.

When an input is permanently too large and the relevant feature is enabled, Klar can split role work into bounded chunks. Prompt projection uses the minimum Resume/job fields needed and a bounded job-description excerpt.

Decision boundary

Every change remains pending until the person accepts, rejects, or edits it. Evidence-blocked changes are rejected at:

- proposal normalization/audit;
- accept/edit decision;
- readiness calculation; and
- export assembly.

This repeated enforcement is deliberate defense in depth. A UI shortcut must not bypass it.

Writing artifacts

Recruiter messages and cover letters use explicit language, tone, evidence, request, and formality contracts. Generated output must remain grounded in the canonical Resume and selected job snapshot. Deterministic checks catch structural and evidence-contract failures but do not prove every sentence is factually correct or professionally effective.

The recruiter message remains a workspace artifact and is not included in the current application ZIP. The ZIP includes a Resume DOCX and semantic cover-letter DOCX only when current cover-letter provenance and readiness are valid.

Format and provenance contracts

Current identifiers include:

Artifact	Contract
Packet	klar-packet-v1
Application content	klar-application-content-v1

Artifact	Contract
Writing prompt	klar-writing-v2.6.0
Resume DOCX generator	klar-resume-docx-v2.6.0.1
Cover-letter DOCX	klar-cover-letter-docx-v1
Application ZIP	klar-application-packet-zip-v1

Historical artifacts without provenance remain explicitly `historical:unversioned`. Packet-level labels must not make mixed old/new artifacts appear to have one generator version. Each artifact carries its own format, generator, prompt/model, language, time, and source-state provenance where applicable.

Document generation

The current Resume DOCX and browser-print PDF share a cross-compatible A4 template using Arial and normalized margins/spacing. The 2.6.0.1 hotfix aligns the Resume export contract across supported formats. Cover letters use a semantic DOCX structure rather than a visual screenshot or canvas representation.

The Resume design lab under `src/resume/designLab.ts` is an experimental evaluation surface, disabled by default and not exposed in the publicly distributed product. Its variants do not change the production exporter unless a separately reviewed change promotes them.

Exports are assembled locally. A one-download ZIP supports mobile workflows, but the downloaded file leaves Klar's local protection boundary.

Human review and release truth

No generated application is auto-sent. Users must review facts, dates, company and role names, language, salutation, claims, and destination requirements before use.

The repository's human writing gate remains **HOLD** until the required real, independent, professionally bilingual evaluations exist. Machine checks, model self-evaluation, synthetic fixtures, or attractive rendering cannot substitute for that evidence. Thresholds and reviewer design are documented in [Test Strategy and Release Gates](#).

Change checklist

A change to Resume or application generation must review:

- canonical schema and migration impact;
- input extraction bounds and data minimization;
- consumer-side structured validation;
- evidence linking and blocked-claim behavior;
- retry/token/deadline bounds;
- English and German independence;
- packet/history/provenance compatibility;

- DOCX, print/PDF, and ZIP behavior;
- interruption recovery;
- accessibility and mobile download behavior; and
- automated plus applicable human gates.

Provider architecture is documented in [AI, Provider, Prompt, and Evidence Architecture](#). Operational recovery is in [Operations Runbooks](#).

AI and Models

1 controlled documents in this section.

AI, Provider, Prompt, and Evidence Architecture

Current AI execution paths, provider configuration, structured-output controls, prompt minimization, evidence rules, budgets, and local-model foundation.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

AI in Klar is an explicitly invoked assistant for extraction, explanation, and application writing. It is not the authority for career facts, source truth, eligibility, deterministic rank, or release approval. This page separates the production AI path from the provider-neutral and local-model foundation so capability is not overstated.

Architecture layers

Layer	Current role	Authority
Deterministic product logic	Filtering, requirements, ranking, evidence checks, normalization, readiness, document assembly	Authoritative for executable product constraints
Production LLM client (src/llm/)	Groq-compatible chat, prompt construction, extraction, optional analysis, tailoring, and writing	Current application generation path
Provider-neutral foundation (src/ai/)	Capability contracts, request schemas, base/precision/writer adapters, cloud/local providers, registry	Verified foundation and preview integration; not yet the normal generation router
Cloud provider	Wraps the existing chat client	Uses configured external provider path
Local provider	Wraps desktop generation IPC	Desktop preview capability only
Consumer validator	Parses and validates each capability-specific result	Required authority before product state changes
Human reviewer	Accepts career facts and application claims	Final authority for user-facing application content

Current production provider path

The default engine is Groq using an OpenAI-compatible chat interface. The default quality and fast model choices are the GPT-OSS 120B and 20B variants configured by Klar. When the default engine and a Worker URL are configured, requests use the Worker's Groq relay. A custom OpenAI-compatible endpoint is called directly from the renderer.

Custom base URLs may use HTTP or HTTPS; the UI warns about mixed-content and security implications. A custom engine can declare that no API key is required. This flexibility is intended for user-controlled compatible endpoints and does not make an endpoint trusted.

Provider settings are stored as non-secret configuration. Credential storage follows the rules in [Data Architecture, Vault, and Backup](#).

Explicit invocation and data minimization

Deterministic search, filtering, ranking, taxonomy, deduplication, cache, tracking, and document layout do not require AI.

When the user invokes an AI action, the capability constructs a bounded projection rather than sending the entire database. Examples include:

- extracted Resume text for initial structuring;
- selected Resume sections and a bounded job-description excerpt for tailoring;
- normalized job fields and deterministic evidence for explanation;
- selected packet language, facts, job context, and request for cover-letter or recruiter-message writing.

Prompts must not include credentials, vault envelopes, unrelated jobs, operational events, backup files, or hidden application history. Data is sent to the configured provider only for that invocation. Provider-side retention and training behavior are governed by that provider's terms and configuration, not by local Klar deletion.

Structured output

For the default supported Groq models, the chat client requests strict JSON Schema output. Custom or other compatible models use JSON-object mode where strict schema support is not known. The transport can make one compatibility retry when the provider rejects schema generation, and it can recover a valid structured payload from supported failed-generation responses.

Transport success is never sufficient. Each consumer validates its own response shape, bounds, indices, enum values, evidence references, and semantic invariants. Invalid responses fail closed or leave the deterministic result intact.

The provider-neutral foundation adds common request limits:

- at most 32 messages;
- at most 120,000 total message characters;
- at most 32,000 schema characters;
- a timeout between 1 and 120 seconds; and
- capability-specific output limits.

These are safety ceilings, not recommended prompt sizes. Individual consumers use tighter projections.

Budget and retry policy

The production budget controller starts with conservative client-side assumptions of 8,000 tokens per minute and 30 requests per minute. Token estimation is approximately characters divided by 3.6 and reserves the requested maximum output before dispatch. A rolling in-memory 60-second window:

- rejects a request that is permanently too large;
- waits for temporary headroom when the relevant queue flag permits; and
- learns a lower token-per-minute limit from supported provider quota errors.

The configured daily-token field is not currently enforced. Client estimates do not replace provider-authoritative quotas, multi-client coordination, or an abuse-control service. Retries are capability-specific and bounded; generation must not recurse indefinitely.

Evidence architecture

Evidence exists to keep fluent writing subordinate to reviewed facts.

Stage	Control
Resume extraction	Prompt requires copy-only behavior; response is normalized and reviewed before canonical storage
Job analysis	Deterministic job/Resume facts exist before optional provider explanation
Tailoring	Provider returns change proposals tied to original content and evidence references
User decision	Each change is accepted, rejected, or edited; unsupported changes remain blocked
Writing	Capability receives a bounded fact inventory and job snapshot, not permission to invent missing facts
Export	Readiness and evidence checks run again before an artifact is assembled
Release	Automated evidence checks plus the applicable independent human gate

No prompt instruction can guarantee truth on its own. The decisive controls are constrained inputs, structured consumers, immutable source facts, evidence links, repeated decision/export gates, provenance, and human review.

Production prompt responsibilities

Every prompt contract must state:

- the capability and non-goals;
- the allowed source facts;

- the required language and output schema;
- how missing or ambiguous information is represented;
- a prohibition on unsupported claims and fabricated metrics;
- evidence/reference requirements;
- size and output bounds; and
- what the consumer will do when validation fails.

Prompt and generator versions are artifact provenance. Changing wording that can alter output meaning requires a version decision, fixtures, consumer validation, regression review, and applicable human evaluation.

Provider-neutral foundation

`src/ai/` defines capabilities for structured job extraction, evidence selection, recruiter messages, cover letters, and Resume bullet revision. It includes base, precision, and writer adapter slots and a registry that requires an explicit provider selection. It does not silently switch from local to cloud or vice versa.

The foundation is used by tests and the desktop local-runtime panel. Current application call sites for Resume extraction, tailoring, cover letters, and related generation still call the existing `chatComplete` production path. Therefore, a verified local model is not yet selectable as the provider for normal application workflows in 2.6.0.1. Treat local generation as a foundation/developer-preview capability, not a completed product privacy claim.

Local-model path

On desktop, the local provider sends a validated request across the narrow preload bridge. Electron main owns the loopback runtime URL and bearer credential; the renderer receives neither. The pinned runtime supports JSON-object plus schema constraints, disabled thinking, one active request, cancellation, and bounded diagnostics.

The currently documented base package is Qwen3.5-9B Q4_K_M with pinned artifact provenance. It is not bundled or automatically downloaded. Precision and writer adapter slots exist in the contract but their release evidence remains **HOLD** until genuine packages and benchmarks are supplied. See [Electron Desktop and Local Runtime](#).

Failure behavior

- Provider unavailable: preserve deterministic results and current packet state; show an actionable retry path.
- Invalid JSON/schema: reject the response; use only the bounded compatibility retry defined by the client.
- Missing AI row in a batch: retain the local row without a fabricated AI score.
- Quota exceeded: do not cache a false result; expose a safe budget/provider error.
- Interrupted desktop generation: mark the request cancelled or interrupted and keep the prior reviewed artifact.
- Evidence failure: block the proposed change or export, regardless of provider confidence.

Change and review requirements

Any provider, model, prompt, capability, schema, budget, or fallback change must review privacy disclosure, network path, credentials, input projection, output validation, failure isolation, evidence, locale, provenance, diagnostics redaction, automated tests, and applicable human gates.

Current limitations are consolidated in [Known Gaps and Risk Register](#); runbook actions are in [Operations Runbooks](#).

Platform

2 controlled documents in this section.

Worker API and Network Security

Cloudflare Worker routes, request contracts, upstream allowlists, CORS, SSRF controls, limits, secrets, observability, and failure behavior.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

The `klar-proxy` Cloudflare Worker makes a small set of browser-incompatible or credential-bearing requests possible without becoming a general backend or open proxy. Its central control is reconstruction: callers choose among known Klar routes and bounded parameters, never an arbitrary upstream URL.

Deployment baseline

The Worker entry point is `worker/src/index.ts`. Its deployment configuration pins the runtime compatibility baseline, enables the compatibility features used by the implementation, and restricts production browser origins. Operational logs must remain free of personal and secret payloads.

Secrets such as Groq or Adzuna credentials are deployment secrets, not source-controlled variables. Credential selection is atomic: incomplete credential sets are rejected rather than combined.

Route catalog

Route family	Method	Upstream/purpose	Principal controls
Health	GET	Worker health and capability response	No personal input; safe status only
BA jobs	GET	BA Jobsuche API	Fixed upstream family, reconstructed request, no-store
Adzuna jobs	GET	Adzuna jobs API	Fixed upstream family, atomic credential selection, incoming credential query fields removed
Source Fabric	GET	Allowlisted source retrieval	Fixed host/path catalog, HTTPS, redirect revalidation, type and byte bounds
AI model discovery	GET	Configured provider model discovery	Credential required, fixed upstream operation, bounded response
AI generation	POST	Structured generation	JSON contract, request and response bounds, credential required, redirects rejected
Any known route	OPTIONS	CORS preflight	Origin policy and allowed header/method response

Unknown routes return 404. A non-GET method outside the Groq completion route returns 405.

CORS and caller boundary

Production configuration uses an explicit browser-origin allowlist. CORS remains a browser-read control rather than an authentication mechanism, so independent abuse controls are required for any relay exposed to the public internet.

The Worker accepts only the headers required by each route family. Origin policy does not replace authentication, rate limiting, provider quotas, or monitoring.

BA relay

The BA relay uses a fixed upstream family and adds the upstream's required public-client identification. Search and detail enrichment currently depend on separate upstream contracts. The Worker does not accept a caller-selected BA host.

Responses are relayed with no-store semantics and application-facing errors are content-safe. Consistent resource bounds are not yet enforced across every relay; closing that gap is a public release objective.

Adzuna relay

The Adzuna relay removes credential fields from the incoming query before rebuilding the request. Credentials must come from one complete configured source; incomplete sets are rejected rather than combined. The upstream host and route family are fixed.

Like the BA relay, this path still needs the same independently verified resource-limit policy used by the most constrained relay paths. Worker protection should not depend only on downstream parsing.

Source Fabric retrieval

Fabric uses a fixed hostname-to-path-prefix allowlist mirrored by tests against the client registry. It:

- requires a recognized connector/host mapping;
- rebuilds an HTTPS URL;
- bounds path length;
- manually follows a small, fixed number of redirects and revalidates each destination;
- screens destinations against private and local network ranges;
- applies a hard response-size maximum;
- validates the expected content type;
- streams with a byte bound;
- strips XML document type/entity declarations from non-JSON text; and
- never executes upstream JavaScript.

The Worker allowlist materially limits server-side request-forgery reach. Network-destination validation still needs additional defense in depth across address formats, name resolution, and redirect handling; the fixed host catalog reduces but does not eliminate that risk.

The roadmap also calls for stronger structured-data bounds, query validation, active-content sanitization, cache-key hardening, and client/session abuse controls. Not all of those are independently enforced at the Worker boundary in 2.6.0.1; downstream parsers and fixed routes provide partial defense.

Groq relay

Only model discovery and chat completions are exposed. Both require a bearer token supplied for the request. The Worker does not persist the token and forwards it only in the upstream authorization header.

For chat completions, the Worker requires a bounded JSON object, constrains the upstream response, rejects redirects, and returns no-store responses. It does not provide a general OpenAI-compatible proxy, account store, or model-hosting service.

The current Worker does not yet provide a complete identity-aware abuse-control layer for every relay. Production operations should combine provider-side controls with privacy-safe monitoring and must never log prompts, Resume content, job descriptions, or credentials.

Error and logging policy

Worker errors exposed to the application must communicate category and recovery without including upstream secrets, request authorization, prompt bodies, personal content, stack traces, or unsafe upstream HTML. Logs and traces should contain route, status class, latency, bounded size, connector ID, and request correlation tokens only.

Upstream source content is untrusted even after a 2xx response. Client consumers still validate required fields, content structure, URLs, provenance, and schema.

Security review checklist

A new route, host, or path must answer:

- Why can the browser not safely perform the request directly?
- Is the upstream finite and owned/authorized for the intended public retrieval?
- Can a caller influence scheme, authority, credentials, redirect, path, query, or headers beyond the minimum contract?
- What are the method, timeout, redirect, request-size, response-size, content-type, parsing-depth, and rate limits?
- How are private-network destinations, name resolution, address formats, active content, and cache poisoning handled?
- What is logged and how is it proven content-free?
- What does the client do on partial, malformed, blocked, or stale responses?
- Which contract and security tests fail if the allowlist drifts?

Deployment and incident procedures are in [Operations Runbooks](#). Cross-system threats are in [Security, Privacy, and Threat Model](#).

Electron Desktop and Local Runtime

Desktop renderer isolation, IPC contract, local model process lifecycle, signed package trust, diagnostics, packaging, and experimental limitations.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

The Electron build adds an isolated, verified local-model runtime without granting the web renderer general desktop privileges. The 2.6.0.1 desktop capability is experimental and its binaries are not publicly distributed; its security architecture is implemented and tested, but public signing, adapter evidence, and normal-workflow local-provider integration are not complete.

Process and privilege model

Process	Privilege	Responsibility
Renderer	Browser sandbox	Shared React UI and local IndexedDB workflows
Preload	Context-isolated bridge	Publish a frozen, narrow, type/size-checked API
Electron main	Privileged desktop authority	Window policy, IPC validation, package trust, process lifecycle, diagnostics
llama-server	Child process on loopback	Execute one local generation request at a time

BrowserWindow uses `nodeIntegration: false, contextIsolation: true, sandbox: true`, and `webSecurity: true`. Webviews and insecure content are not enabled. The minimum window is 960 × 700.

Renderer loading and content security

In development, the renderer URL must be loopback HTTP with path `/` or `/index.html`. A packaged build loads the local application file. The production content-security policy allows scripts only from self, blocks frames and objects, and permits network connections only to self and HTTPS. Development adds the bounded loopback and WebSocket needs of Vite.

All permission requests are denied. Same-window navigation is limited to the trusted renderer. External opening accepts only credential-free HTTPS URLs that are not loopback. The IPC sender must be the main frame and

exactly match the trusted development origin or packaged file boundary.

The current package retains a compatibility-related Electron fuse exception while using the existing packaged-renderer loading model. A future custom application protocol should remove that exception through a separately tested migration.

Preload bridge

The bridge exposes only:

- redacted system information;
- runtime status, start, and stop;
- AI generation and cancellation; and
- redacted diagnostics.

It exposes no generic `invoke`, `filesystem`, `shell`, `process`, `environment`, or `path` API. Preload validation improves error feedback; main-process validation is authoritative and repeated to protect against a compromised renderer.

Every IPC change must constrain channel, sender, frame, object keys, string sizes, enum values, request ID, timeout, schema, and lifecycle state. Unknown properties are rejected rather than forwarded to the runtime.

Runtime lifecycle

The bundled runtime version is pinned to `llama.cpp b10199`. On start, main:

- 1 verifies the selected model package and compatibility;
- 2 locates the platform-specific bundled runtime;
- 3 creates a random API credential in a mode-0600 temporary file;
- 4 starts `llama-server` with an allowlisted environment;
- 5 binds `127.0.0.1` on an ephemeral port;
- 6 disables the runtime web UI and external access, limits parallelism to one, and applies supported model/adapter arguments;
- 7 polls health until ready or the bounded startup fails; and
- 8 deletes the credential file after warmup while retaining the secret only in main memory.

The renderer never receives the port authorization credential. One generation is active at a time. Cancellation, crash, and window close move the runtime through explicit states and reject affected requests. Stop first requests graceful termination and then uses forced termination after bounded waits; close and application quit always clean up the child process.

The pinned runtime uses thinking disabled and JSON-object plus schema constraints for structured generation. Compatibility behavior is tied to the runtime pin and must be revalidated before an upgrade.

Model package trust

Managed packages live under the Electron user-data directory at `models/packages/{artifactId}`.

Package schema version 1 uses:

- a manifest describing artifact, runtime, app range, prompt contract, platform, base/adaptor role, and provenance;
- an exact checksum map; and
- an Ed25519 signature over canonical JSON containing checksums and manifest.

Verification rejects:

- untrusted key IDs or invalid signatures;
- missing, extra, non-regular, or symlink files;
- path traversal;
- size or SHA-256 mismatch;
- manifest/checksum key mismatch;
- provenance source-artifact mismatch; and
- incompatible app, runtime, prompt schema, platform, or adapter relation.

Installation uses staging and an atomic final rename and does not replace an existing package implicitly.

The compatibility baseline is Klar $\geq 2.6.0$ $< 2.7.0$, llama.cpp b10199, and prompt schema `klar-generation-v1`. A 2.7 release must deliberately update or preserve this range; changing only the application version can make every current package incompatible by design.

Trust keys

Development can load a public key from `KLAR_MODEL_PUBLIC_KEY_PEM`. Packaged builds trust only `resources/trust/model-signing-public.pem` with the production key ID. The private signing key must never be committed, packaged, written to diagnostics, or supplied to the application at runtime.

The current documented base-model package pins the Qwen3.5-9B Q4_K_M upstream artifact, revision, size, and digest. The model and adapter files are not bundled or downloaded automatically. Base-package preparation is supported; precision and writer adapter claims remain **HOLD** until genuine artifacts and benchmark evidence exist.

Memory and support tiers

Desktop tiering is based on physical memory:

- below 12 GiB: unsupported;
- 12 GiB to below 24 GiB: minimum tier; and
- 24 GiB or more: recommended.

This is a compatibility guard, not a guarantee of latency or output quality. Model size, platform, other processes, and runtime upgrades still affect behavior.

Diagnostics and privacy

Desktop diagnostics are memory-only and capped at 200 events. Redaction removes known sensitive keys, credentials, emails, paths, and URL query/hash data and applies generic secret patterns. Diagnostics are operational aids, not a formally complete data-loss-prevention system. The issue-report flow shows a redacted preview and opens the external issue destination; it never submits automatically.

Packaging and release state

The experimental build configuration produces:

- macOS Apple Silicon ZIP with ad-hoc signature; and
- Windows x64 portable package.

CI builds and tests these packages but deliberately does not publish them. Production configuration targets signed/notarized macOS arm64+x64 DMG/ZIP and signed Windows x64 NSIS, with a draft GitHub release provider. Production signing remains **HOLD** until real credentials, notarization, provenance, hashes, smoke results, and approval are present.

Electron fuses are explicit. Packaged smoke and fuse verification test the artifact after packaging; source tests alone do not prove the final fuse or ASAR state.

Current product limitation

The desktop runtime panel can install/verify, start/stop, generate/cancel, and inspect diagnostics through the provider-neutral foundation. Normal Resume extraction, tailoring, cover-letter, recruiter-message, and related application call sites still use the production cloud-compatible client. Klar 2.6.0.1 must not claim that ordinary application generation automatically stays on-device.

Change checklist

Review window options, CSP, navigation, permissions, IPC schema, sender validation, secret handling, runtime arguments, environment, port binding, cancellation, crash recovery, package canonicalization, signature/trust roots, compatibility, fuses, packaging, smoke tests, and logs for every desktop change.

Release controls are in [Test Strategy and Release Gates](#); incident steps are in [Operations Runbooks](#); unresolved desktop items are in [Known Gaps and Risk Register](#).

Engineering

1 controlled documents in this section.

Engineering Setup and Change Guide

Authoritative development setup, local workflows, verification commands, and safe change procedures for each Klar subsystem.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

This guide provides the minimum reproducible setup and the review path for changes that can affect data, source integrity, generated claims, network trust, or desktop privilege. It does not authorize a release; release gates are defined in [Test Strategy and Release Gates](#).

Authoritative baseline

Always inspect the Klar repository root directly before designing a release or build guide. Do not reconstruct current behavior from older guides or roadmap prose. Record the current commit, package.json release values, worktree state, and the exact evidence used.

Current release metadata has two deliberate representations:

- npm/electron package version: 2.6.0-1;
- product release: 2.6.0.1.

Use klarRelease for product documentation and emitted version.json; packaging tools may use the semver-compatible package version.

Prerequisites

- Node.js ^20.19.0 or >=22.12.0; CI uses Node 22.
- npm and the committed lockfile.
- A modern browser with IndexedDB and Web Crypto.
- A deployed/configured Klar Worker for live proxied sources and reliable hosted Groq access.
- Platform-specific desktop runtime and trust material only when doing desktop package work.

Use npm ci for a clean, lockfile-reproducible install. Do not regenerate the lockfile or upgrade pinned desktop tools as a side effect of unrelated work.

Local startup

- 1 Install dependencies with `npm ci`.
- 2 Copy `.env.example` to the ignored `.env.local` only if it does not already exist.
- 3 Set `VITE_WORKER_URL` to the intended deployed Worker URL without a trailing slash.
- 4 Start the web renderer with `npm run dev`.
- 5 For the Electron development shell, use `npm run desktop:dev`.

Never commit real provider credentials, vault/backup passwords, private signing keys, `.env.local`, runtime binaries, model files, or personal test documents. Use synthetic fixtures unless controlled human-evidence procedures explicitly permit anonymized/consented material.

Fast feedback commands

Command	Purpose	What it does not prove
<code>npm run typecheck</code>	App TypeScript consistency	Worker, runtime behavior, build output, source health
<code>npm run worker:types:check</code>	Wrangler-generated binding type freshness	Worker TypeScript semantics or deployment health
<code>npm run worker:check</code>	Worker TypeScript consistency	Network policy in a deployed zone
<code>npm test</code>	All sorted <code>test/*.test.ts</code> regressions	Production bundle, audits, human gates, packaged binaries
<code>npm run build</code>	Production web renderer and <code>version.json</code>	Desktop-relative asset loading or deployment
<code>npm run build:desktop</code>	Relative-base renderer for Electron	Signed/package runtime correctness
<code>npm run bundle:check</code>	Enforce generated bundle byte ceilings	Runtime performance or accessibility
<code>npm run worker:dry-run</code>	Validate Worker packaging/configuration without deploying	Live Worker/source correctness
<code>npm run qa</code>	Complete repository automated release gate	Dependency audits, real source verification, human review, signing

`npm run qa` writes build artifacts under `dist`; use focused non-writing checks during documentation-only or read-only reviews.

Change procedure

For every change:

- 1 identify the authoritative domain and invariant in [Codebase and Domain Catalog](#);
- 2 inspect current implementation, configuration, fixtures, and tests;
- 3 state current behavior, intended behavior, non-goals, and migration/compatibility impact;
- 4 add the narrowest domain change and regression tests;
- 5 verify failure, cancellation, retry, and recovery paths-not only the happy path;

- 6 update format, prompt, model, parser, schema, or decision versions when meaning changes;
- 7 update current KB pages and known gaps in the same release change; and
- 8 run proportionate checks locally, followed by the full CI/release sequence before merge or distribution.

Preserve unrelated worktree changes. Do not use destructive reset/checkout commands to make a review appear clean.

Data or schema change

- Assign one authority and classify the data as sensitive, credential, operational, or cache.
- Add a new Dexie version; do not rewrite a shipped migration.
- Keep unrelated content migrations out of additive store changes.
- Authenticate/decrypt before mutation and use one transaction for authority changes.
- Update vault content/credential schema, backup modes, historical import, wipe, retention, and locked-state behavior.
- Invalidate derived caches when semantics change.
- Test empty database, upgrade fixtures, vault on/off, wrong passphrase, corrupt data, interruption, backup round trip, and rollback behavior.

See [Data Architecture](#), [Vault](#), and [Backup](#).

Resume, packet, or export change

- Preserve Resume schema version 2 unless a coordinated migration is approved.
- Ensure import normalization never invents facts.
- Keep Profile derived.
- Maintain per-language packet independence and per-artifact provenance.
- Enforce evidence at proposal, decision, readiness, and export boundaries.
- Render and inspect DOCX/PDF output; run semantic text and cross-compatibility checks.
- Version the affected generator/format contract and keep historical rows visibly historical.
- Re-run applicable writing human evidence before making new quality claims.

See [Resume and Application Generation](#).

Career ranking change

- Freeze normalized input and an asOf time.
- Keep hard filters, eligibility facts, deterministic rank, preference fit, and posting confidence separate.
- Preserve unknown rather than inferring mismatch.
- Update ranking/requirements/explanation versions when feature meaning or weighting changes.
- Test input-order invariance, stable hashes, source failure, missing fields, hard mismatches, and unbounded visible results.

- Compare against the frozen 2.5.5 baseline and use only eligible bilingual human evidence for release claims.

See [Career Discovery and Ranking](#).

Connector or source change

- Verify official ownership and permitted public retrieval behavior.
- Set a finite host/path allowlist, parser version, query/pagination contract, timeout, retry eligibility, content type, byte cap, cache/freshness, fallback, and kill switch.
- Capture licensed or safe fixtures for inventory, zero inventory, malformed, timeout, blocked, removal, redirect, and duplicate cases.
- Preserve per-field provenance and distinguish candidate from verified support.
- Update both client registry and Worker allowlist tests.
- Prove one connector cannot delay the session past its deadline or erase earlier valid batches.

See [Flexible Work and Source Fabric](#) and [Worker API and Network Security](#).

Provider, model, or prompt change

- Make provider selection explicit; never add a silent cloud/local fallback.
- Minimize the input projection and document which data leaves the device.
- Bound request, output, time, retry, and budget behavior.
- Add consumer-side validation; transport schema mode is not enough.
- Tie generated edits and claims to evidence and preserve human review.
- Update prompt/model/generator provenance and invalid-response fixtures.
- Test quota, empty body, malformed structured output, cancellation, and partial batch results.
- Run machine benchmarks as diagnostics and the applicable held-out human gate for release claims.

See [AI, Provider, Prompt, and Evidence Architecture](#).

Worker change

- Add only a finite product route; never accept a caller-supplied upstream URL.
- Define method, origin behavior, host/path, credentials, request/response bounds, redirects, content types, timeout, errors, logs, and abuse controls.
- Revalidate every redirect and keep untrusted upstream content out of logs.
- Run Worker type, allowlist/security tests, generated-types check, and dry run.
- Deploy Worker configuration separately from static Pages and verify the production origin list.

Electron or model-package change

- Keep renderer code free of Electron/Node imports.
- Add only narrow IPC and duplicate validation in preload/main, with main authoritative.

- Review CSP, navigation, permissions, fuses, secret ownership, process cleanup, and diagnostics.
- Preserve signed canonical manifest/checksum verification, path/symlink rejection, compatible trust root, staging, and atomic install.
- Revalidate the pinned runtime before changing version or flags.
- Run packaged smoke and fuse checks on the exact archive; source tests are insufficient.
- Never treat CI's ephemeral trust key or ad-hoc signature as production trust.

See [Electron Desktop and Local Runtime](#).

Documentation change

Use all required frontmatter fields, the canonical term Resume, and /docs/<slug> internal links. Distinguish current, target, historical, draft, and deprecated statements. Follow [Contributing Documentation](#), [Documentation Style Guide](#), and [Source of Truth](#).